MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Done by: st. gr. FAF-242, Temciuc Adelina

Report

on laboratory work no. 4
Operating Systems
Topic: The xv6 Shell from the Outside

Checked by:
Reiţman Patricia, university assistant
DISA, FCIM, UTM

Chişinău – 2026

AIM OF THE WORK

Purpose of the laboratory work

The purpose of this laboratory work is to understand, by experiment, what a command shell must do and what a full shell such as bash adds on top of that minimum. The xv6 shell (`user/sh.c`, about 500 lines) is used side by side with bash, and a small program that prints its own arguments shows which part of the system expands *, removes quotes and splits the command line into words.

The laboratory work aims to:

- update the sleep program from Lab 2 for the current xv6, where the system call `sleep()` was renamed to `pause()`;
- run the same command lines in bash and in the xv6 shell and compare redirection, pipes, grouping, sequences, background jobs, globbing, variables, quotes, PATH and history;
- write a program that prints argc and argv for xv6 and for Linux and use it to find out who expands * and removes quotes;
- read the source code of the xv6 shell and find the command kinds it knows, where each kind is executed and how cd is handled.

THEORETICAL BACKGROUND

The shell as an ordinary user program

A shell is not part of the kernel. It is an ordinary user program that reads a line, splits it into words, recognises special characters and then asks the kernel to create processes with `fork()` and to load programs with `exec()`. Everything the user sees as "the command line" (pipes, redirection, variables, wildcards) is implemented by the shell itself on top of a few system calls: fork, exec, wait, open, close, dup, pipe and chdir.

The xv6 shell

In xv6 the whole user interface is the program `user/sh.c`. It parses a line into a tree of commands of five kinds: simple commands (EXEC), redirections (REDIR), pipes (PIPE), lists separated by ; (LIST) and background commands (BACK); parentheses group commands into a block. The tree is executed by the recursive function `runcmd()` in a child process. The xv6 shell has no variables, no wildcards, no quote handling, no search path and no history.

Arguments, argv and exec

A program receives its command line as an array of strings, argv, with argc elements; `argv[0]` is the name the program was started with. The kernel does not interpret these strings: `exec(path, argv)` only copies them onto the stack of the new program. Any transformation of the line (wildcard expansion, variable substitution, quote removal, word splitting) must therefore happen in the shell before exec is called.

Built-in commands

Some commands cannot be separate programs because they must change the state of the shell process itself. The most important one is cd: the current directory is a property of a process, so if cd ran in a child process only the child would change directory and the change would be lost when the child exits.

PART 1. FIX SLEEP FOR THE CURRENT XV6

All the work for this laboratory was done on my current laptop, Fedora Linux 44, as my own user adelina, with the xv6 tree in `~/xv6-riscv`; all the figures in this report were taken there, in the Ptyxis terminal.

1.1 Which name does the tree use

In August 2025 the xv6 authors renamed the system call `sleep()` to `pause()`. To find out which name the local tree uses, the declaration was searched for in `user/user.h`:

```
cd ~/xv6-riscv
grep -n "pause\|sleep" user/user.h
```

The output was `25:int pause(int);`, so the tree is a current one: only pause is declared and the old call `sleep()` from Lab 2 no longer compiles.



Figure 1.1 – The grep result in user.h

1.2 The updated sleep.c

The call in `user/sleep.c` was changed from `sleep()` to `pause()`. The program checks that exactly one argument was given, prints a usage message to standard error (file descriptor 2) otherwise and exits with status 1:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // was sleep() before Aug 2025
  exit(0);
}
```

The line `$U/_sleep\` was added to UPROGS in the Makefile, directly after `$U/_sync\`, starting with a TAB and with nothing after the backslash (line 154 in Figure 1.2).

```
adelina@192:~/xv6-riscv$ cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // was sleep() before Aug 2025
  exit(0);
}
adelina@192:~/xv6-riscv$ grep -n -B1 '_sleep' Makefile
153-    $U/_sync\
154:    $U/_sleep\
adelina@192:~/xv6-riscv$
```

Figure 1.2 – The updated sleep.c and the UPROGS line

1.3 Build, boot and test

xv6 was built and started with `make qemu`. At the $ prompt, `sleep 20` paused the shell for 20 ticks (about two seconds) and then the prompt returned; `sleep` without an argument printed the usage message. QEMU was left with Ctrl-A X, which prints `QEMU: Terminated`.

```
adelina@192:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3
format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ sleep 20
$ sleep
usage: sleep <ticks>
$ QEMU: Terminated
adelina@192:~/xv6-riscv$
```

Figure 1.3 – sleep running in xv6, with and without an argument

PART 2. BASH VERSUS THE XV6 SHELL

2.1 Running the commands in both shells

Every command from the table was run twice: once in bash, in an empty test folder that contains only the file README (`mkdir ~/shtest && cd ~/shtest && touch README`), and once at the xv6 $ prompt after `make qemu`. The results are summarised in table 2.1.

Table 2.1 – The same commands in bash and in the xv6 shell

Command	Tests	bash	xv6	Why
`echo hello > f ; cat < f`	redirection	hello	hello	Both shells open the file and connect it to stdout or stdin before exec.
`ls | grep READ`	pipe	README	README 2 2 2441	Both create a pipe; xv6 ls also prints type, inode and size.
`(echo a; echo b) | wc`	grouping, list	2 2 4	2 2 4	Both support ( ) and ;. Only the spacing of wc differs.
`echo one ; echo two`	sequence	one two	one two	Both run the commands one after another.
`echo bg &`	background	[1] 53770 bg [1]+ Done	bg	Both run it without waiting; only bash has job control, so only bash prints the job number and "Done".
`ls *`	globbing	f README	ls: cannot open *	bash replaces * with the file names; xv6 passes the character * to ls unchanged.
`echo $HOME`	variables	/home/adelina	$HOME	xv6 has no variables, so $HOME is an ordinary word.
`echo "a b"`	quotes	a b	"a b"	bash removes the quotes and keeps one word; xv6 keeps the quotes and splits the text into two words.
`mkdir d ; cd d ; ls ..`	PATH	d f README	exec cd failed, then the listing of /	xv6 treats cd as a built-in only at the start of a line; here it tries to exec a program cd, which does not exist.
arrow up, Tab, Ctrl+R	history, completion	recalls, completes and searches commands	exec ^[[A failed	xv6 has no line editor, history or completion; the escape code of the arrow is run as a command name.

In bash (Figure 2.1), Tab completed `ls ../RE` to `ls ../README`, the up arrow recalled that command again, and Ctrl+R opened the reverse history search: after typing `echo o` it found `echo one ; echo two`, as shown in the last line of the figure.

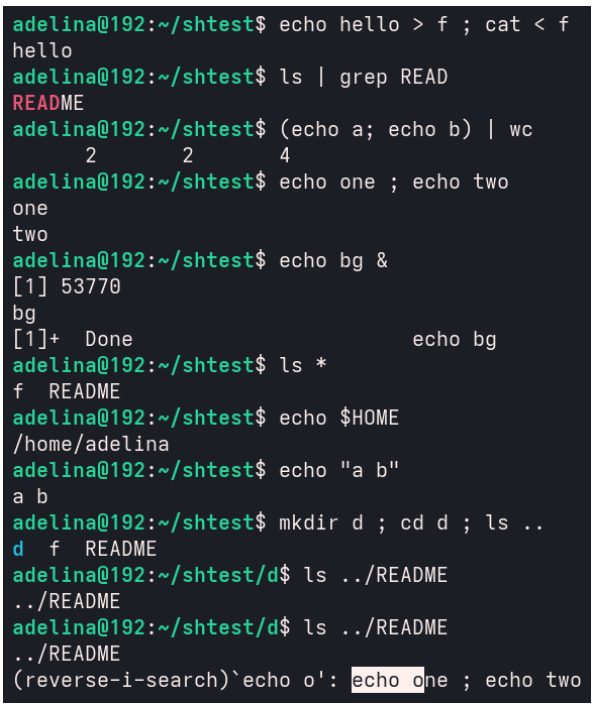


Figure 2.1 – The commands run in bash

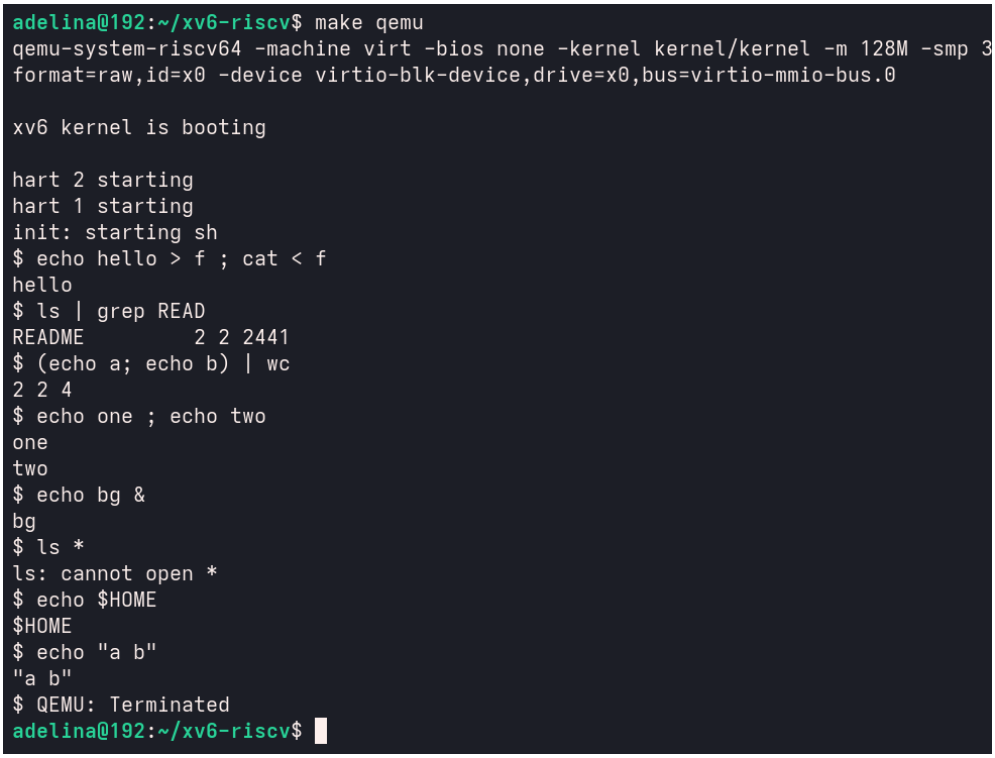


Figure 2.2 – The same commands run in xv6

Figure 2.3 shows the second part of the xv6 run (after a new `make qemu`; the file system image keeps the file f from the first run). Because cd was not at the beginning of the line, xv6 printed `exec cd failed`, stayed in / and `ls ..` listed the root directory, where the programs sleep and args, the new file f (6 bytes, "hello" and a newline) and the new empty directory d (32 bytes, the entries . and ..) appear.

Then the keys up arrow, Tab and Ctrl+R were pressed, followed by Enter. The xv6 console has no line editor, so it simply echoes the characters it receives. The up arrow is the escape sequence ESC [A; when it is echoed, the terminal itself executes it and moves the cursor one line up, so the following text overwrote the end of the line of d. The shell then took ESC [A as a program name and printed `exec ^[[A failed`, which is the word "failed" that appears in that line of the figure.

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ mkdir d ; cd d ; ls ..
exec cd failed
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35944
echo           2 4 34840
forktest       2 5 17776
grep           2 6 43368
init           2 7 35192
kill           2 8 34744
ln             2 9 34536
ls             2 10 42024
mkdir          2 11 34792
rm             2 12 34776
sh             2 13 57152
stressfs       2 14 35616
usertests      2 15 201600
grind          2 16 50416
wc             2 17 36912
zombie         2 18 34088
logstress      2 19 36760
forphan        2 20 35552
dorphan        2 21 35016
sync           2 22 34192
sleep          2 23 34520
args           2 24 34504
console        3 25 0
f              2 26 6
d     failed   1 27 32
$ QEMU: Terminated
adelina@192:~/xv6-riscv$
```

Figure 2.3 – ls .. after the failed cd, and the history keys in xv6

In a third run, `cd d` was typed alone on a line, so the built-in worked, and `ls` was tried inside d. It failed with `exec ls failed`. Giving the path of the program, `/ls` or `../ls`, made it work and listed the two entries of d.

```
adelina@192:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3
format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ cd d
$ ls
exec ls failed
$ /ls
.              1 27 32
..             1 1 1024
$ ../ls
.              1 27 32
..             1 1 1024
$ QEMU: Terminated
adelina@192:~/xv6-riscv$
```

Figure 2.4 – ls failing after cd d, and the working alternative

2.2 Answers to the questions

1. Which features do both shells have, and which ones only bash has?

Both shells can run programs with arguments, redirect input and output with <, > and >>, connect commands with pipes |, run sequences with ;, group commands with parentheses, start commands in the background with & and change the directory with the built-in cd.

Only bash has wildcards (globbing), variables and $ expansion, quote removal, a search path (PATH), job control (job numbers, jobs, fg, "Done" messages), && and ||, command history, Tab completion and history search, and a full scripting language with if, for and functions.

2. Why does ls stop working after cd d, and how can it be run anyway?

The xv6 programs live in the root directory /, and the xv6 shell has no PATH: it calls exec with exactly the name that was typed. `exec("ls")` is a relative path, so the kernel looks for ls in the current directory. In / it is found, but in /d there is no file ls, so exec fails. The program can still be run by giving its location: `/ls` (absolute path) or `../ls` (relative path), as shown in Figure 2.4.

3. Which missing feature would you miss most, and which would be easiest to add?

In daily work I would miss history and Tab completion the most, because they save a lot of typing, together with wildcards. The easiest feature to add is a simple PATH: if `exec(argv[0])` fails and the name does not contain a /, the shell can try again with "/" in front of the name. That is only a few lines in the EXEC case of `runcmd()` and would fix the ls problem from question 2.

PART 3. WHO EXPANDS *? A PROGRAM THAT PRINTS ITS ARGV

3.1 The args program for xv6

The program `user/args.c` prints the number of arguments and every argument between square brackets, so that spaces and quotes inside an argument are visible. The line `$U/_args\` was added to UPROGS after `$U/_sleep\`.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

```
adelina@192:~/xv6-riscv$ cat user/args.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
adelina@192:~/xv6-riscv$ grep -n -B1 '_args' Makefile
154-    $U/_sleep\
155:    $U/_args\
adelina@192:~/xv6-riscv$
```

Figure 3.1 – args.c and the UPROGS line

3.2 The same program for Linux

For Linux, the file `args_linux.c` includes `<stdio.h>` instead of the two xv6 headers and ends with `return 0` instead of `exit(0)`. It was compiled with `gcc -o args args_linux.c` in the folder `~/lab4`, which also contains the files args.c, args_linux.c and sleep.c.

```
#include <stdio.h>

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  return 0;
}
```

3.3 Running the same lines in both shells

The following lines were run in xv6 and, with `./args`, in bash:

```
args one   two three
args *.c "x y"
args $HOME
args 'single quoted'
```

```
adelina@192:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3
format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ args one   two three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
$ args 'single quoted'
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
$ QEMU: Terminated
adelina@192:~/xv6-riscv$
```

Figure 3.2 – args output in xv6

```
adelina@192:~/lab4$ ls
args.c  args_linux.c  sleep.c
adelina@192:~/lab4$ gcc -o args args_linux.c
adelina@192:~/lab4$ ./args one   two three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
adelina@192:~/lab4$ ./args *.c "x y"
argc = 5
argv[0] = [./args]
argv[1] = [args.c]
argv[2] = [args_linux.c]
argv[3] = [sleep.c]
argv[4] = [x y]
adelina@192:~/lab4$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/adelina]
adelina@192:~/lab4$ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
adelina@192:~/lab4$
```

Figure 3.3 – args_linux output in bash

3.4 Answers to the questions

1. Both outputs for args *.c "x y". How many arguments did each program receive?

In xv6:

```
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
```

In bash:

```
argc = 5
argv[0] = [./args]
argv[1] = [args.c]
argv[2] = [args_linux.c]
argv[3] = [sleep.c]
argv[4] = [x y]
```

In xv6 the program received 4 arguments: *.c unchanged, and "x and y" as two separate words with the quote characters still in them. In bash it received 5 arguments: the three .c files of the folder instead of *.c, and x y as one argument, without quotes.

2. Who turns *.c into file names and removes the quotes?

The shell. The program is the same in both cases and only prints what it receives, and the kernel only copies the argv array given to exec. The difference comes only from the shell: bash expands *.c, substitutes $HOME (it printed /home/adelina) and removes quotes before calling

exec, while the xv6 shell does none of this, so the program sees the raw words ($HOME, 'single and quoted').

3. What happened to the extra spaces in one two three?

They disappeared in both shells. The shell uses spaces only as separators between words, so any number of spaces between two words produces just one boundary, and the program received exactly three arguments: one, two and three. To keep the spaces they must be quoted in bash ("one two"); in xv6 this is impossible, because it does not handle quotes.

PART 4. READ THE SHELL

4.1 Commands executed

Inside `~/xv6-riscv` the following commands were run:

```
wc -l user/sh.c
grep -n "#define" user/sh.c    # the kinds of commands
grep -n "case " user/sh.c     # where each kind is run
grep -n "cd" user/sh.c | head
```

The file `user/sh.c` has 499 lines. Lines 8–12 define the five command kinds.

```
adelina@192:~/xv6-riscv$ wc -l user/sh.c
499 user/sh.c
adelina@192:~/xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
adelina@192:~/xv6-riscv$
```

Figure 4.1 – The command kinds defined in sh.c

The cases on lines 75–125 are in `runcmd()`, which executes each kind. Lines 284–294 are in `gettoken()`, which recognises the special characters, and lines 469–493 are in `nulterminate()`, which finishes the parsed tree.

```
adelina@192:~/xv6-riscv$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
adelina@192:~/xv6-riscv$
```

Figure 4.2 – Where each kind is run

`grep -n "cd"` finds only the error message on line 170, because the code tests the characters c and d separately. Searching for chdir shows the whole block in `main()`.

```
adelina@192:~/xv6-riscv$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
adelina@192:~/xv6-riscv$ grep -n -B3 -A4 "chdir" user/sh.c
166-    if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
167-      // Chdir must be called by the parent, not the child.
168-      cmd[strlen(cmd) - 1] = 0; // chop \n
169:      if (chdir(cmd + 3) < 0)
170-        fprintf(2, "cannot cd %s\n", cmd + 3);
171-    } else {
172-      if (fork1() == 0)
173-        runcmd(parsecmd(cmd));
adelina@192:~/xv6-riscv$
```

Figure 4.3 – The code that handles cd

Finally, sh.c was searched for any code that handles * or $, that is for the character constants '*' and '$'. Both searches found nothing (grep exit code 1); the only special characters the shell knows are listed in the arrays whitespace and symbols on lines 268–269.

```
adelina@192:~/xv6-riscv$ grep -nF "'*'" user/sh.c ; echo "exit code: $?"
exit code: 1
adelina@192:~/xv6-riscv$ grep -nF "'$'" user/sh.c ; echo "exit code: $?"
exit code: 1
adelina@192:~/xv6-riscv$ grep -nE "^char (whitespace|symbols)" user/sh.c
268:char whitespace[] = " \t\r\n\v";
269:char symbols[] = "<|>&;()";
adelina@192:~/xv6-riscv$
```

Figure 4.4 – No code for * or $ in sh.c

4.2 Answers to the questions

1. The five kinds of command and the character that produces each

- EXEC (1) – a simple command, a program name with its arguments; it needs no special character;
- REDIR (2) – redirection, produced by <, > or >>;
- PIPE (3) – a pipe, produced by |;
- LIST (4) – a sequence, produced by ;;
- BACK (5) – a background command, produced by &.

Parentheses ( ) only group commands into a block; they do not create a sixth kind.

2. Where is cd handled, and why is it not run as a separate program?

cd is handled in `main()`, on lines 166–170, before the shell forks: if the line starts with the characters c, d and a space, the shell calls `chdir(cmd + 3)` itself and prints `cannot cd` if it fails. It is not a separate program because every other command runs in a child process created by `fork()`. A child that called chdir would change only its own current directory, and the change would be lost when the child exits, so the shell would stay where it was. The comment in the code says it directly: "Chdir must be called by the parent, not the child". This is also why cd in the middle of `mkdir d ; cd d ; ls ..` failed: there it is parsed as a normal EXEC command.

3. Is there any code that handles * or $? What does that explain about Part 2?

No. The tokenizer only knows the whitespace characters " \t\r\n\v" and the symbols <|>&; (); there is no code for *, $, quotes or PATH. Every other character is just part of a word. This explains the results of Part 2 and Part 3: `ls *` passed the character * to ls, `echo $HOME` printed $HOME, `echo "a b"` kept the quotes and split the text into two words, and ls could not be found outside /.

CONCLUSION

In this laboratory work the xv6 shell was studied from the outside, by using it side by side with bash and by reading its source code.

In the first part, the sleep program from Lab 2 was updated for the current xv6, where the system call is called `pause()`. After adding it to UPROGS it built and worked: `sleep 20` paused the shell and sleep without an argument printed the usage message.

In the second part, the same commands were run in both shells. Both support redirection, pipes, grouping, sequences, background commands and cd, which are exactly the features that need only fork, exec, pipe, dup and chdir. Everything that is about text processing of the command line (wildcards, variables, quotes, PATH, history and completion) exists only in bash. The xv6 shell also showed that cd is a built-in only at the beginning of a line, and that without PATH a program in / must be called with its full path from any other directory.

In the third part, the args program proved that the program and the kernel do not change the arguments: for `args *.c "x y"` the xv6 program received 4 raw words, while the bash version received 5 arguments with the file names expanded and the quotes removed. The shell alone is responsible for expansion, quote removal and word splitting.

In the fourth part, reading `user/sh.c` (499 lines) showed the five command kinds EXEC, REDIR, PIPE, LIST and BACK, the function `runcmd()` that executes them, the special handling of cd in the parent process and the absence of any code for * or $, which explains all the differences observed in Part 2.

REFERENCES

1. Cox R., Kaashoek F., Morris R. xv6: a simple, Unix-like teaching operating system. MIT, 2024. [online]. Available: https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf
2. mit-pdos/xv6-riscv – source code. [online]. Available: https://github.com/mit-pdos/xv6-riscv
3. bash(1) – Linux manual page. [online]. Available: https://man7.org/linux/man-pages/man1/bash.1.html
4. GNU Bash Reference Manual. Free Software Foundation. [online]. Available: https://www.gnu.org/software/bash/manual/bash.html